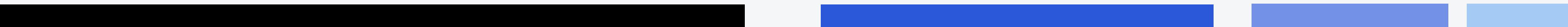


Lecture-3

Dynamic Programming

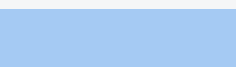
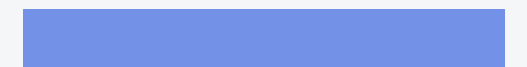
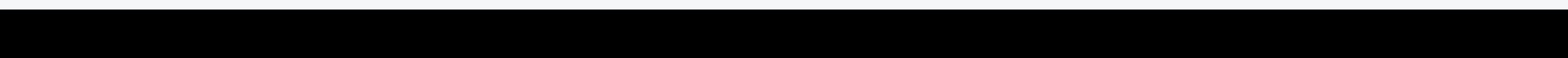


Agenda

01. Policy Evaluation
02. Policy Iteration
03. Value Iteration
04. Asynchronous Dynamic Programming
05. Generalized Policy Iteration
06. Bootstrapping

01.

Policy Evaluation



Policy Evaluation/Prediction

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

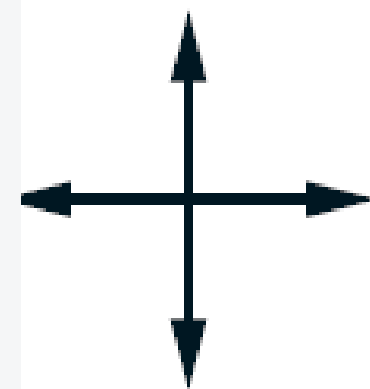
$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Policy Evaluation/Prediction

Consider the 4×4 gridworld shown below.



actions

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R_t = -1$
on all transitions

Policy Evaluation/Prediction

$k = 0$

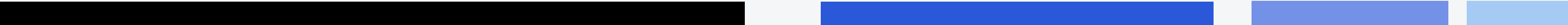
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0



Advantage Function

- The advantage function measures how much better an action is compared to the policy's average action at that state.

$$A(s, a) = \underbrace{Q(s, a)}_{\substack{\text{q value for action a} \\ \text{in state s}}} - \underbrace{V(s)}_{\substack{\text{average} \\ \text{value} \\ \text{of that} \\ \text{state}}}$$

- $A^\pi(s, a) > 0$: action is better than the current policy's average.
- $A^\pi(s, a) < 0$: action is worse.
- $A^\pi(s, a) = 0$: action is as good as the policy's expected value.

Policy Improvement

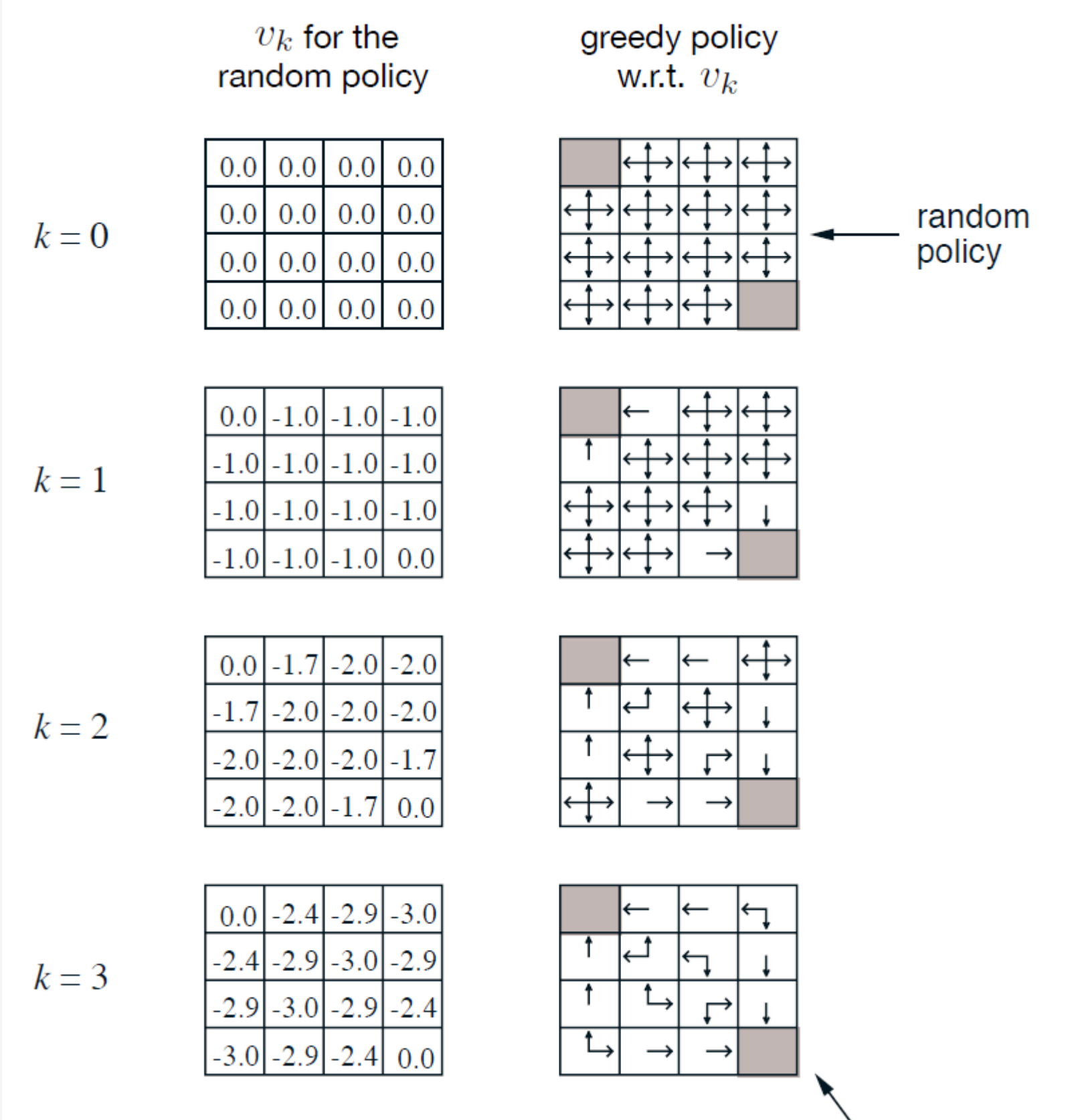
theorem. Let π and π' be any pair of deterministic policies such that, for all $s \in \mathcal{S}$,

$$q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s). \quad (4.7)$$

Then the policy π' must be as good as, or better than, π . That is, it must obtain greater or equal expected return from all states $s \in \mathcal{S}$:

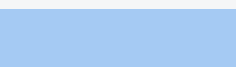
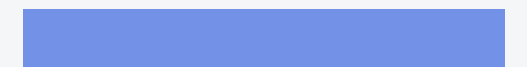
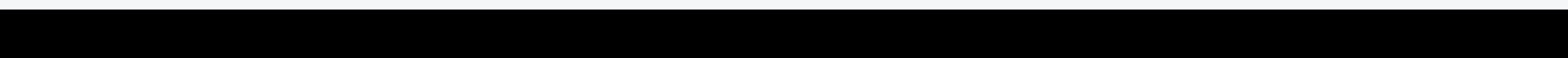
$$v_{\pi'}(s) \geq v_{\pi}(s). \quad (4.8)$$

Policy Improvement



02.

Policy Iteration



Policy Iteration

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*,$$

- Each policy is guaranteed to be a strict improvement over the previous one (unless it is already optimal).
- Because a finite MDP has only a finite number of policies, this process must converge to an optimal policy and optimal value function in a finite number of iterations.

Policy Iteration

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ *true*

For each $s \in \mathcal{S}$:

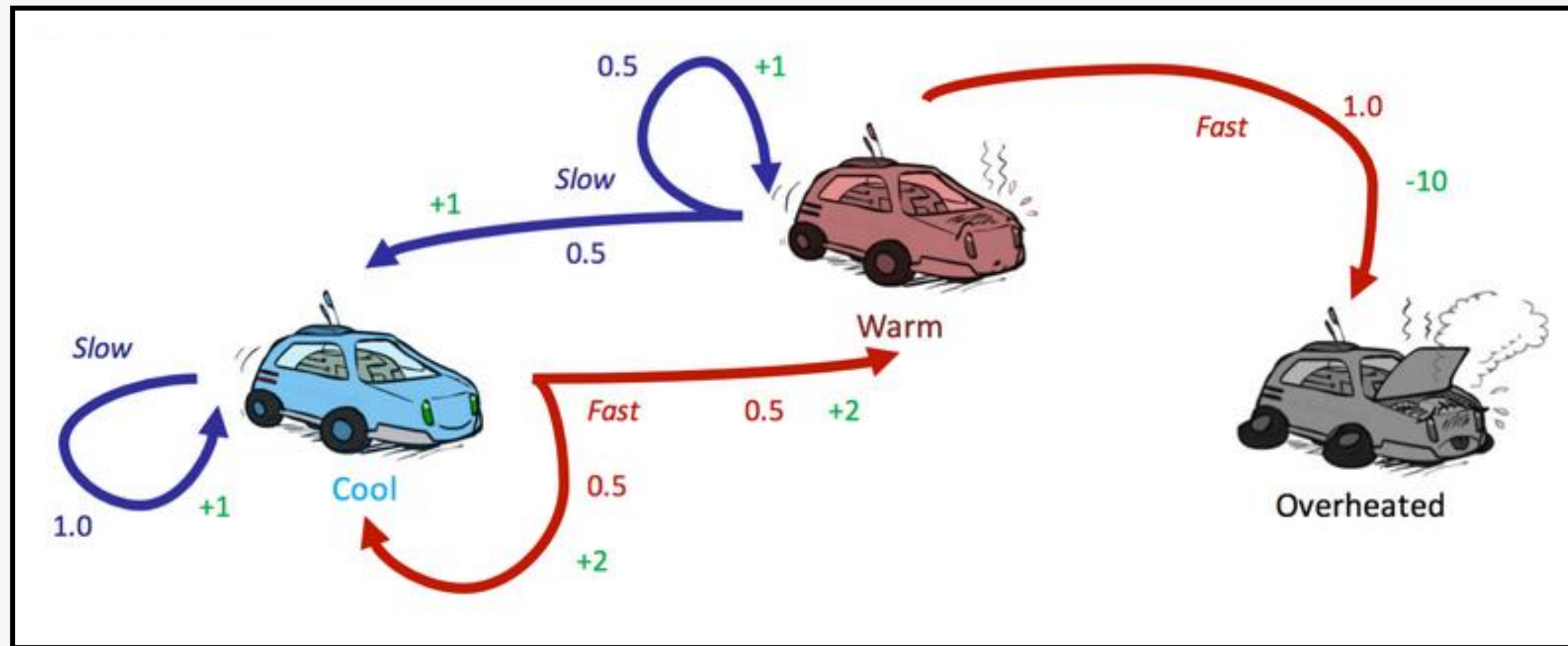
old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Policy Iteration

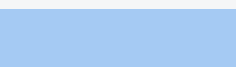
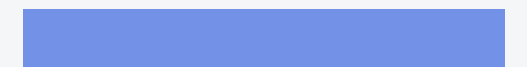
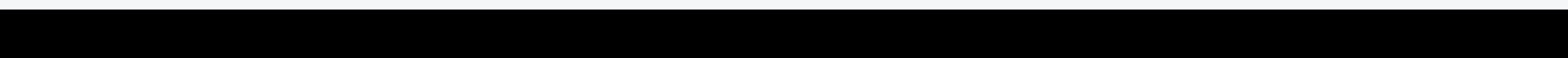


Policy Iteration

- One drawback to policy iteration is that each iteration involves policy evaluation which may itself be a protracted iterative computation requiring multiple sweeps through the state set.
- If policy evaluation is done iteratively, then convergence exactly to v occurs only in the limit.

03.

Value Iteration



Value Iteration

- In value iteration, policy evaluation is stopped after just one sweep (one update of each state).
- It can be written as a particularly simple update operation that combines the policy improvement and truncated policy evaluation steps:

$$\begin{aligned} v_{k+1}(s) &\doteq \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')], \end{aligned}$$

Value Iteration

- The value iteration is obtained simply by turning the Bellman optimality equation into an update rule.

$$\begin{aligned} v_*(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')], \text{ or} \end{aligned}$$

- Also note how the value iteration update is identical to the policy evaluation update except that it requires the maximum to be taken over all actions.

Value Iteration

Value Iteration, for estimating $\pi \approx \pi_*$

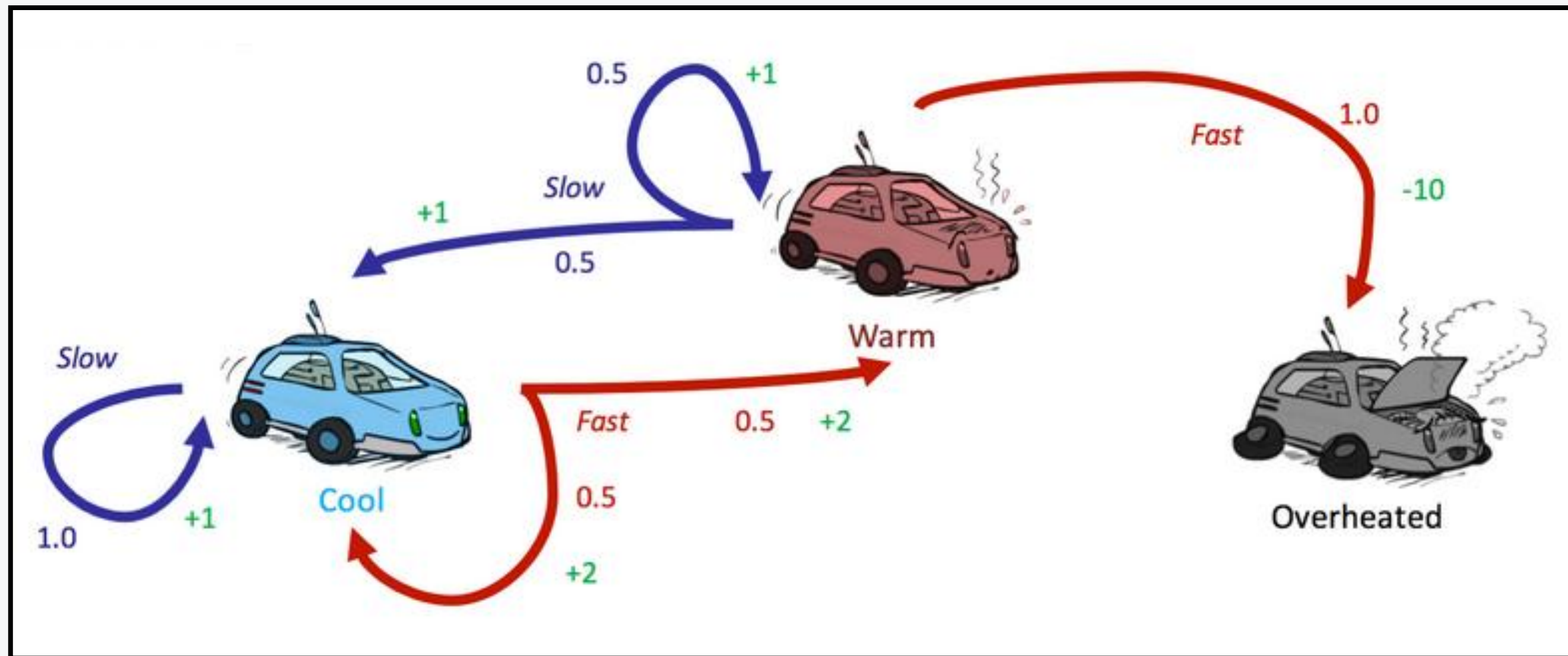
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

- | $\Delta \leftarrow 0$
- | Loop for each $s \in \mathcal{S}$:
- | $v \leftarrow V(s)$
- | $V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$
- | $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
- until $\Delta < \theta$

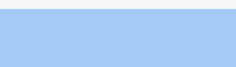
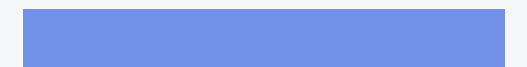
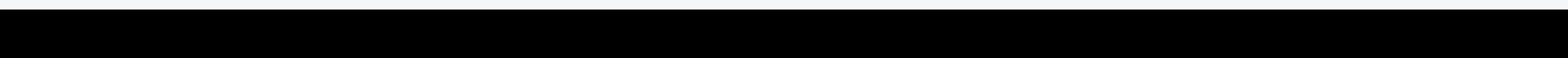
Output a deterministic policy, $\pi \approx \pi_*$, such that
$$\pi(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

Value Iteration



04.

Asynchronous DP



Asynchronous DP

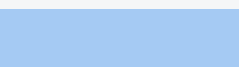
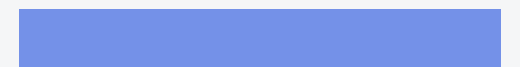
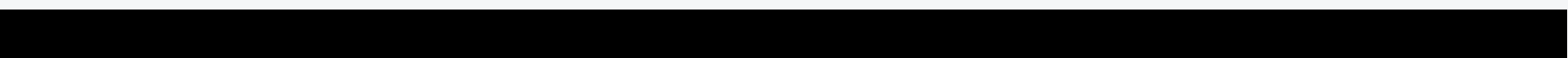
- A major drawback to these DP methods is that they require sweeps of the state set.
- If the state set is very large, then even a single sweep can be prohibitively expensive.

Asynchronous DP

- Asynchronous DP algorithms are in-place iterative DP algorithms that are not organized in terms of systematic sweeps of the state set.
- These algorithms update the values of states in any order whatsoever, using whatever values of other states happen to be available.
- To converge correctly, however, an asynchronous algorithm must continue to update the values of all the states: it can't ignore any state after some point in the computation.

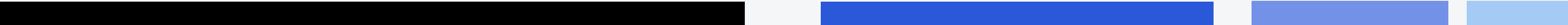
05.

Generalized Policy Iteration

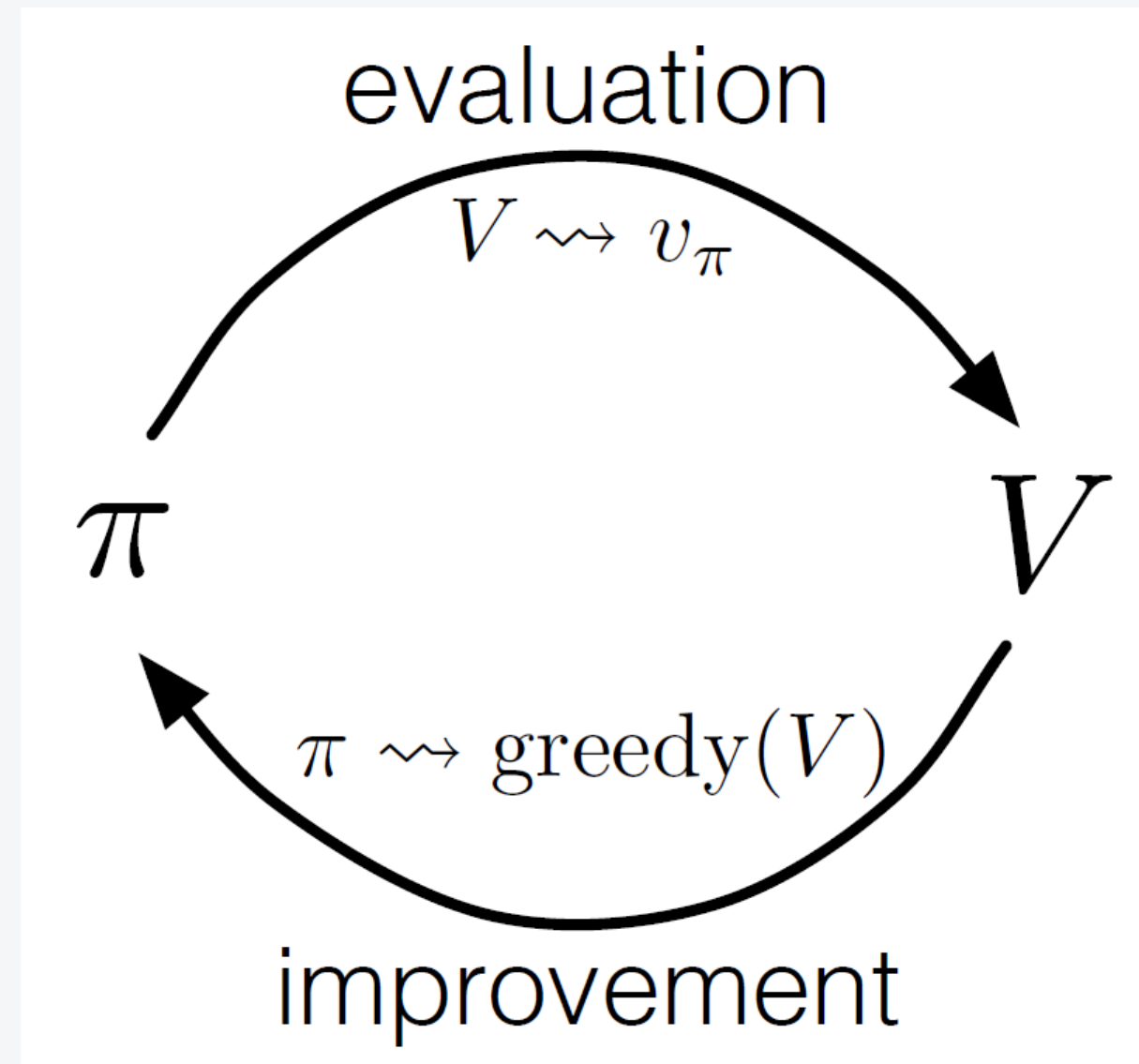


Generalized Policy Iteration

Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Equation + Greedy Policy Improvement	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

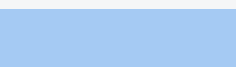
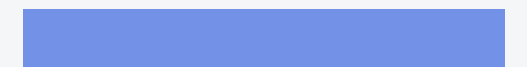
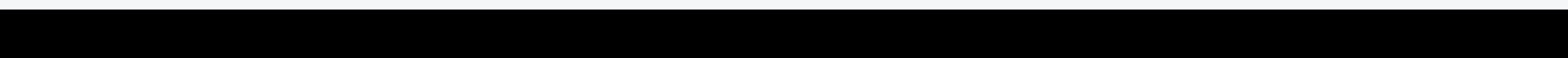


Generalized Policy Iteration



06.

Bootstrapping



Bootstrapping

- DP algorithms update estimates of the values of states based on estimates of the values of successor states.
- That is, they update estimates based on other estimates.
- This idea is bootstrapping.
- Many reinforcement learning methods perform bootstrapping, even those that do not require, as DP requires, a complete and accurate model of the environment.

**Thank
You!**

